

Projeto: DataChat SQL Lite

Assistente Inteligente para Consulta de Dados em Linguagem Natural

Objetivo

Desenvolver uma aplicação inteligente capaz de responder perguntas sobre a base de dados **Brazilian E-Commerce Public Dataset (Olist)** utilizando linguagem natural.

O sistema deverá interpretar perguntas formuladas pelo usuário, gerar automaticamente consultas SQL utilizando um Modelo de Linguagem (LLM), executá-las sobre um banco de dados e apresentar os resultados acompanhados de uma explicação em linguagem natural.

O objetivo do projeto é integrar conceitos de **LLMs, Prompt Engineering, bancos de dados relacionais, SQL, Python e desenvolvimento de aplicações**, permitindo que usuários sem conhecimento de SQL explorem bases de dados de maneira intuitiva.

Base de dados

Será a Brazilian E-Commerce Public Dataset by Olist

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

A base contém aproximadamente **100 mil pedidos** realizados entre 2016 e 2018, distribuídos em múltiplas tabelas relacionais.

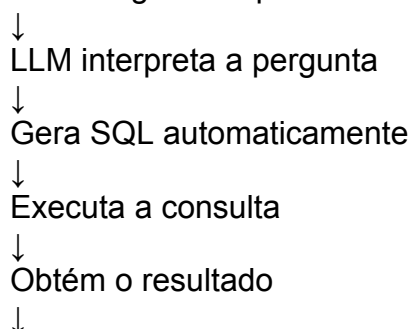
O grupo deverá importar essas tabelas para um banco relacional (preferencialmente DuckDB) e explorar seus relacionamentos por meio de consultas SQL:

- `orders`
- `order_items`
- `products`
- `customers`
- `sellers`
- `payments`
- `reviews`
- `category_translation`
-

Exemplo de Funcionamento

Entrada

Qual categoria de produto teve o maior faturamento em 2018?



Gera uma explicação em linguagem natural

Saída

Pergunta

Qual categoria de produto teve o maior faturamento em 2018?

SQL

```
</> SQL

SELECT
    p.category,
    SUM(oi.price) AS faturamento
FROM order_items oi
JOIN products p
ON oi.product_id = p.product_id
JOIN orders o
ON oi.order_id = o.order_id
WHERE YEAR(o.purchase_date)=2018
GROUP BY p.category
ORDER BY faturamento DESC
LIMIT 1;
```

Resultado

Categoria	Faturamento
Bed & Bath	R\$ 2.148.320

Resposta

Em 2018, a categoria **Bed & Bath** apresentou o maior faturamento da base analisada.

Requisitos Obrigatórios

RF01: Importar automaticamente todos os arquivos CSV da base Olist.

RF02: Criar automaticamente as tabelas no banco de dados.

RF03: Implementar os relacionamentos necessários para consultas envolvendo múltiplas tabelas.

RF04: Permitir consultas em linguagem natural.

RF05: Gerar automaticamente consultas SQL utilizando um LLM.

RF06: Executar as consultas SQL.

RF07:

Exibir:

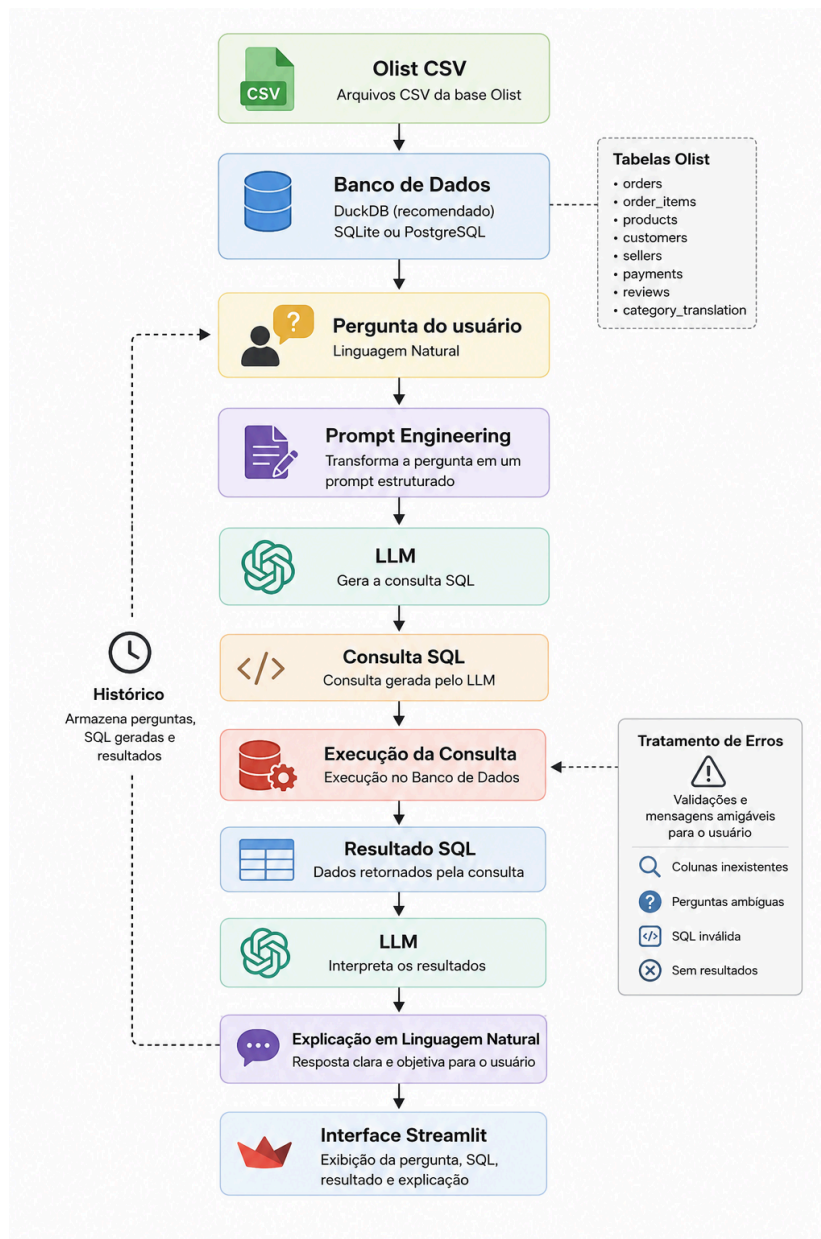
- pergunta do usuário;
- SQL gerada;
- resultado da consulta;
- explicação em linguagem natural.

RF08: Tratar erros simples: perguntas ambíguas; colunas inexistentes; consultas sem resultado; SQL inválida.

RF09: Registrar um histórico das consultas realizadas.

RF10: Permitir consultas envolvendo **JOIN** entre múltiplas tabelas.

Pipeline



Tecnologias

Camada	Tecnologia
Linguagem	Python
Banco de Dados	DuckDB (recomendado), SQLite ou PostgreSQL
LLM	GPT-4o-mini, Qwen, Gemma ou Llama 3 via Ollama
Framework	LangChain ou LlamaIndex
Interface	Streamlit
Visualização	Plotly ou Matplotlib
Versionamento	GitHub

Cronograma (1 mês)

Semana 1 – Preparação da Base

- estudo da base Olist;
- importação dos arquivos CSV;
- criação do banco de dados;
- testes das consultas SQL;
- definição da arquitetura.

Semana 2 – Desenvolvimento do Núcleo

- integração com o banco;
- implementação do módulo Text-to-SQL;
- desenvolvimento do backend;
- primeiros testes com perguntas simples.

Semana 3 – Integração Completa

- integração com o LLM;
- tratamento de erros;
- histórico de consultas;
- testes com consultas envolvendo múltiplas tabelas.

Semana 4 – Finalização

- desenvolvimento da interface;
- gráficos (opcional);
- testes finais;
- documentação;
- gravação do vídeo;
- preparação da apresentação.

Exemplos de Perguntas

Os grupos deverão demonstrar o sistema respondendo perguntas como:

Consultas simples

- Quantos pedidos existem na base?
- Quantos clientes únicos realizaram compras?
- Qual categoria possui mais produtos?

Consultas intermediárias

- Qual categoria teve maior faturamento?
- Qual vendedor realizou mais vendas?
- Qual cidade possui mais clientes?
- Qual foi o ticket médio por estado?
- Qual forma de pagamento é mais utilizada?

Consultas avançadas

- Qual categoria apresentou maior crescimento entre 2017 e 2018?
- Quais vendedores possuem maior avaliação média?
- Quais estados possuem maior tempo médio de entrega?
- Quais categorias possuem maior taxa de avaliações cinco estrelas?
- Qual cidade possui maior faturamento médio por cliente?

Entregáveis

Cada grupo deverá entregar:

- Aplicação funcional (link).
- Código-fonte no GitHub.
- Banco de dados configurado.
- Relatório técnico (10 a 15 páginas).
- Manual de instalação e execução.
- Apresentação (10 a 15 minutos).
- Vídeo demonstrativo (até 5 minutos).

Critérios de Avaliação

Critério	Peso
Funcionamento da aplicação	30%
Geração automática de SQL	20%
Integração com banco de dados	15%
Interface e usabilidade	10%
Qualidade do código	10%
Documentação	10%
Apresentação e demonstração	5%

Desafio Extra (+20% na nota)

O grupo que desejar obter pontuação adicional poderá implementar uma ou mais das seguintes funcionalidades:

- memória de contexto para perguntas sequenciais (ex.: *"E em 2018?"*);
- suporte a consultas analíticas mais sofisticadas (subconsultas e CTEs);
- geração automática de gráficos a partir dos resultados;
- sugestão inteligente de perguntas com base no esquema do banco;
- módulo RAG utilizando a documentação da base Olist (dicionário de dados);
- agente capaz de validar e corrigir automaticamente consultas SQL antes da execução;
- integração do fluxo de consultas com **n8n** para automação.